

ER Modeling: Structure, Constraints & Keys

In this lecture, we deepen our understanding of Entity-Relationship modeling by exploring how to precisely describe the rules governing entity participation, how to represent entities that cannot exist independently, and how to uniquely identify every record in a database. These concepts form the backbone of rigorous database design.

What We'll Cover

This lecture builds on the foundational ER concepts from Lecture 2. We move from simply identifying entities and relationships to specifying the **rules** that govern how they interact — and to handling edge cases that arise in real-world data modeling.

01

Structural Constraints

Cardinality ratios and participation constraints that define how many instances of one entity can relate to another.

02

Weak Entity Types

Entities that depend on a parent entity for their identity and existence — and how to model them correctly.

03

ER Diagram Conventions

Standard notation for entities, relationships, attributes, and constraints in Crow's Foot and Chen notation.

04

Naming Conventions

Best practices for naming entities, relationships, and attributes to ensure clarity and consistency.

05

Introduction to Keys

Unique attributes, primary keys, and how they ensure every record can be individually identified.

Cardinality Ratios

A **cardinality ratio** specifies how many instances of one entity can be associated with instances of another entity through a relationship. It is one of the most important structural constraints in ER modeling because it directly affects how the relationship is implemented in the relational schema.

Four Cardinality Types

1:1 — One to One

Each entity instance relates to at most one instance of the other entity. Example: Employee ↔ Parking Space.

1:N — One to Many

One instance relates to many instances on the other side. Example: Department ↔ Employees.

M:N — Many to Many

Many instances on both sides can relate. Example: Students ↔ Courses.

Why Cardinality Matters

Cardinality is not just a modeling detail — it determines how foreign keys are placed in the relational schema:

- **1:1** — Foreign key can go on either side (often the mandatory side)
- **1:N** — Foreign key goes on the "many" side table
- **M:N** — Requires a separate junction/associative table with two foreign keys

Misidentifying cardinality is one of the most common design errors. Always validate cardinality against real-world business rules, not just assumptions.

Tip: Read the relationship in both directions. "A Department *has* many Employees" and "An Employee *works for* one Department" should both make sense.

Participation Constraints

While cardinality tells us *how many* entities can participate in a relationship, **participation constraints** tell us whether participation is *required* or *optional*. This distinction is critical for enforcing data integrity at the schema level.

Total Participation (Mandatory)

Every instance of an entity **must** participate in the relationship. There can be no "orphan" instances. In ER diagrams, total participation is shown with a **double line** connecting the entity to the relationship.

Example: Every Order *must* be placed by a Customer. An Order cannot exist without being linked to a Customer — this is total participation.

Partial Participation (Optional)

Only *some* instances of an entity are required to participate. Others may exist without being linked to any instance on the other side. Shown with a **single line** in ER diagrams.

Example: Not every Employee is assigned to a Project. An Employee may exist in the database without any Project assignment — this is partial participation.

Combining Cardinality + Participation

Together, these two constraints fully describe a relationship's structural rules. For example, "Each Student must enroll in at least one Course (total) and may enroll in many (M)" is written as **1..M total participation**.

Common Mistake to Avoid

Students often confuse cardinality with participation. Remember: cardinality = *maximum* (how many allowed), participation = *minimum* (how many required). A relationship can be "at most one" (1:1 cardinality) but still be optional (partial participation).

Entities That Can't Stand Alone

A **weak entity type** is an entity that cannot be uniquely identified by its own attributes alone. It depends on a related **owner (or parent) entity** for both its existence and its identity. Without the parent, the weak entity has no meaning in the database.

Characteristics of Weak Entities

- **No independent primary key** — relies on the parent's key plus a partial key (discriminator)
- **Existence-dependent** — if the parent is deleted, the weak entity must also be deleted (cascade delete)
- **Identifying relationship** — the relationship linking it to the parent is shown with a double diamond
- **Partial key (discriminator)** — an attribute that distinguishes weak entities *within* the scope of one parent

Notation in ER Diagrams

- Weak entity: **double rectangle**
- Identifying relationship: **double diamond**
- Partial key attribute: **dashed underline**

Classic Examples

DEPENDENT

A Dependent (spouse, child) cannot exist without an Employee. The partial key might be *DependentName*, but "Alice" only identifies a unique dependent within one Employee's family.

ORDER_ITEM

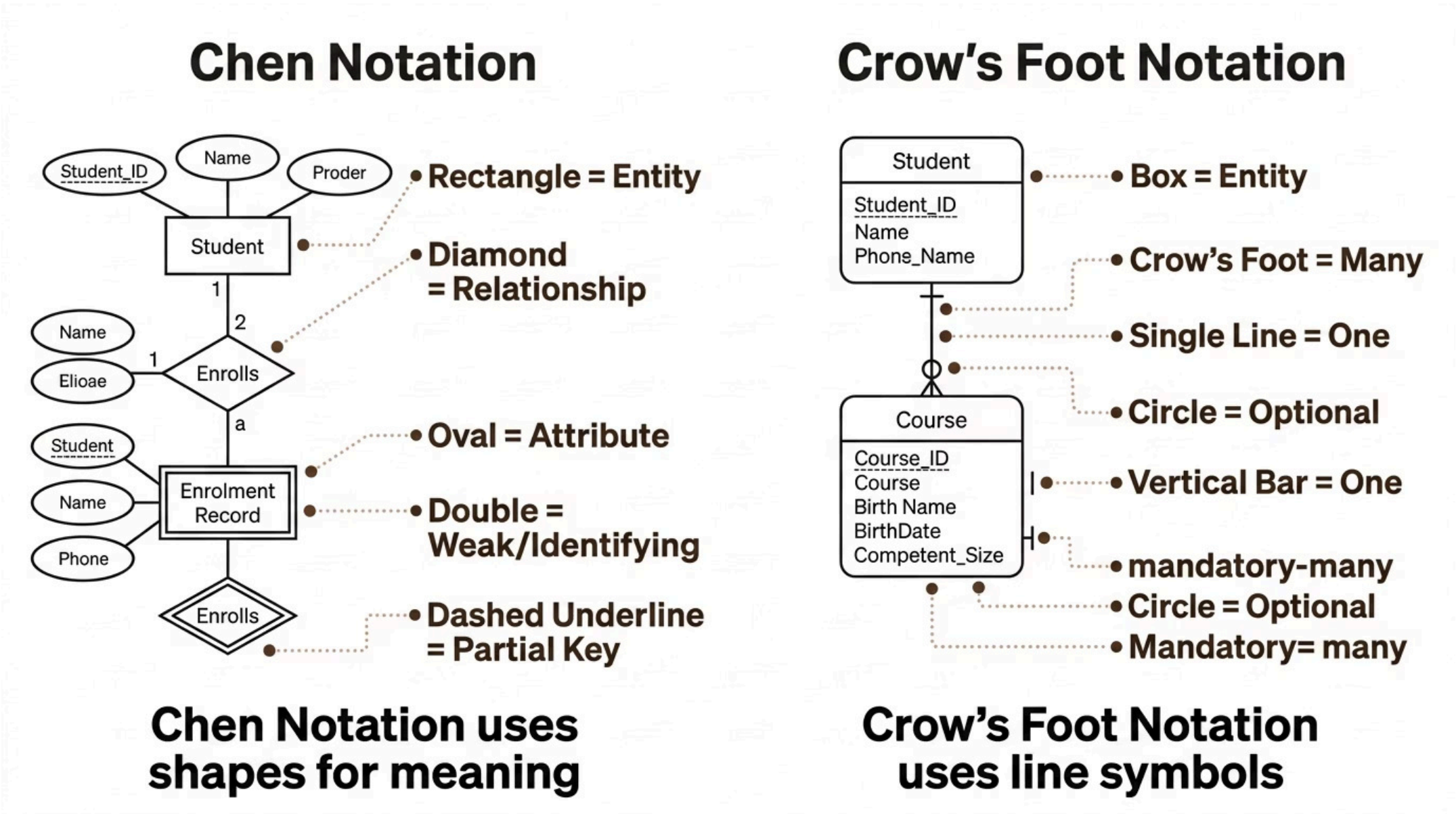
A line item on an Order has no meaning without the Order. Its partial key is often *LineNumber* (1, 2, 3...), which only makes sense relative to a specific Order.

ROOM (in a Hotel)

Room 101 only uniquely identifies a room within a specific Hotel. The full key is (HotelID, RoomNumber).

Reading and Drawing ER Diagrams

ER diagrams are the visual language of database design. Understanding the standard notation allows you to communicate complex data models clearly and unambiguously. Two major notations are used in practice: **Chen Notation** (academic, conceptual) and **Crow's Foot Notation** (industry, implementation-focused).



Both notations express the same concepts but differ in visual style. Crow's Foot is more compact and is the standard in tools like MySQL Workbench, SQL Server Management Studio, and dbdiagram.io. Chen notation is clearer for teaching conceptual modeling because it separates relationships as first-class shapes.



Entity (Rectangle)

Represents a real-world object or concept. Strong entities use a single rectangle; weak entities use a double rectangle.



Attribute (Oval)

Describes a property of an entity or relationship. Key attributes are underlined; partial keys use dashed underlines.



Relationship (Diamond)

Represents an association between entities. Identifying relationships (for weak entities) use a double diamond.



Cardinality Symbols

In Crow's Foot: a single line means "one," a crow's foot means "many," a circle means "optional," and a bar means "mandatory."

Consistent, Clear, and Professional Names

Good naming conventions are not just a stylistic preference — they are a critical part of database design. Clear, consistent names reduce ambiguity, prevent errors during implementation, and make schemas maintainable over time. A poorly named attribute can cause confusion for years.

Entity Naming Rules

- Use **singular nouns**: `Employee`, not `Employees`
- Use **PascalCase** or **UPPER_SNAKE_CASE** consistently
- Avoid abbreviations unless universally understood (`Cust` is bad; `Customer` is better)
- Name entities after the *concept*, not the table: `Order`, not `tblOrder`

Attribute Naming Rules

- Use **camelCase** or **snake_case** consistently
- Prefix foreign key attributes with the parent entity name: `departmentID`, not just `deptID`
- Avoid redundant names: inside `Employee`, use `lastName`, not `employeeLastName`
- Use full words: `salary`, not `sal`; `phoneNumber`, not `phoneNum`

Relationship Naming Rules

- Use **active verbs** that describe the association: `WorksFor`, `EnrollsIn`, `Manages`
- Read the relationship name in both directions to verify it makes sense
- Avoid vague names like `Has` or `RelatedTo`

 **Anti-pattern alert:** Naming a table `tblEmployee` and a column inside it `EmployeeID` is redundant. Inside the `Employee` table, the primary key should simply be `employeeID` (or `id` if using a consistent convention across all tables).

Introduction to Keys: Uniquely Identifying Records

A **key** is an attribute (or set of attributes) that uniquely identifies a specific instance of an entity. Without keys, a database cannot guarantee that records are distinguishable — you could have two identical rows with no way to tell them apart. Keys are fundamental to relational database integrity.



Superkey

Any set of attributes that **uniquely identifies** an entity instance. A superkey may contain extra attributes that aren't needed for uniqueness. Example: {SSN, firstName, lastName} is a superkey for Employee — but firstName and lastName are unnecessary.



Candidate Key

A **minimal superkey** — removing any attribute would cause it to lose uniqueness. An entity can have multiple candidate keys. Example: For Employee, both {SSN} and {employeeEmail} could be candidate keys if both are guaranteed unique.



Primary Key (PK)

The **chosen candidate key** used to identify records in the relational schema. Every entity must have exactly one primary key. It must be unique and never null. By convention, it is underlined in ER diagrams.



Alternate Key

Any candidate key that was **not chosen** as the primary key. These are still unique and may be enforced with UNIQUE constraints in SQL. Example: If SSN is the PK, then employeeEmail becomes an alternate key.

Key Selection: Best Practices and Common Pitfalls

Choosing the right primary key is one of the most consequential design decisions in a database. A poor key choice can lead to update anomalies, inefficient queries, and fragile schemas. Here are the key principles for selecting effective primary keys.

What Makes a Good Primary Key?

→ Unique

Must identify exactly one record — no duplicates allowed, ever.

→ Stable

Should never change over time. A person's name can change; their SSN (or a system-generated ID) should not.

→ Minimal

Use the smallest number of attributes possible. Single-attribute keys are simpler than composite keys.

→ Non-null

Every record must have a value. NULL in a primary key violates entity integrity.

Natural vs. Surrogate Keys

Natural keys use real-world data (SSN, email, ISBN). They are meaningful but can change or be reused. **Surrogate keys** are system-generated (auto-increment integers, UUIDs). They have no business meaning but are guaranteed stable and unique.

Recommendation: In most modern systems, use a surrogate key as the primary key and enforce uniqueness on natural key attributes with a UNIQUE constraint. This gives you stability without losing the ability to validate real-world uniqueness.

📌 **Composite keys** use two or more attributes together (e.g., OrderID + LineNumber for OrderItem). Use them when no single attribute is sufficient — but be aware they complicate foreign key references.

Key Takeaways

This lecture covered the structural rules that make ER models precise and implementable. Understanding these concepts is essential before moving to relational schema translation in upcoming lectures.



Structural Constraints

Cardinality ratios (1:1, 1:N, M:N) define maximum participation; participation constraints (total vs. partial) define minimum participation. Together they fully specify relationship rules.



Weak Entities

Weak entities depend on an owner entity for identity and existence. They use a partial key + parent's primary key to form a complete identifier. Shown with double rectangles and double diamonds.



ER Diagram Notation

Chen notation (rectangles, diamonds, ovals) is ideal for conceptual modeling. Crow's Foot notation is the industry standard for implementation. Know both and when to use each.



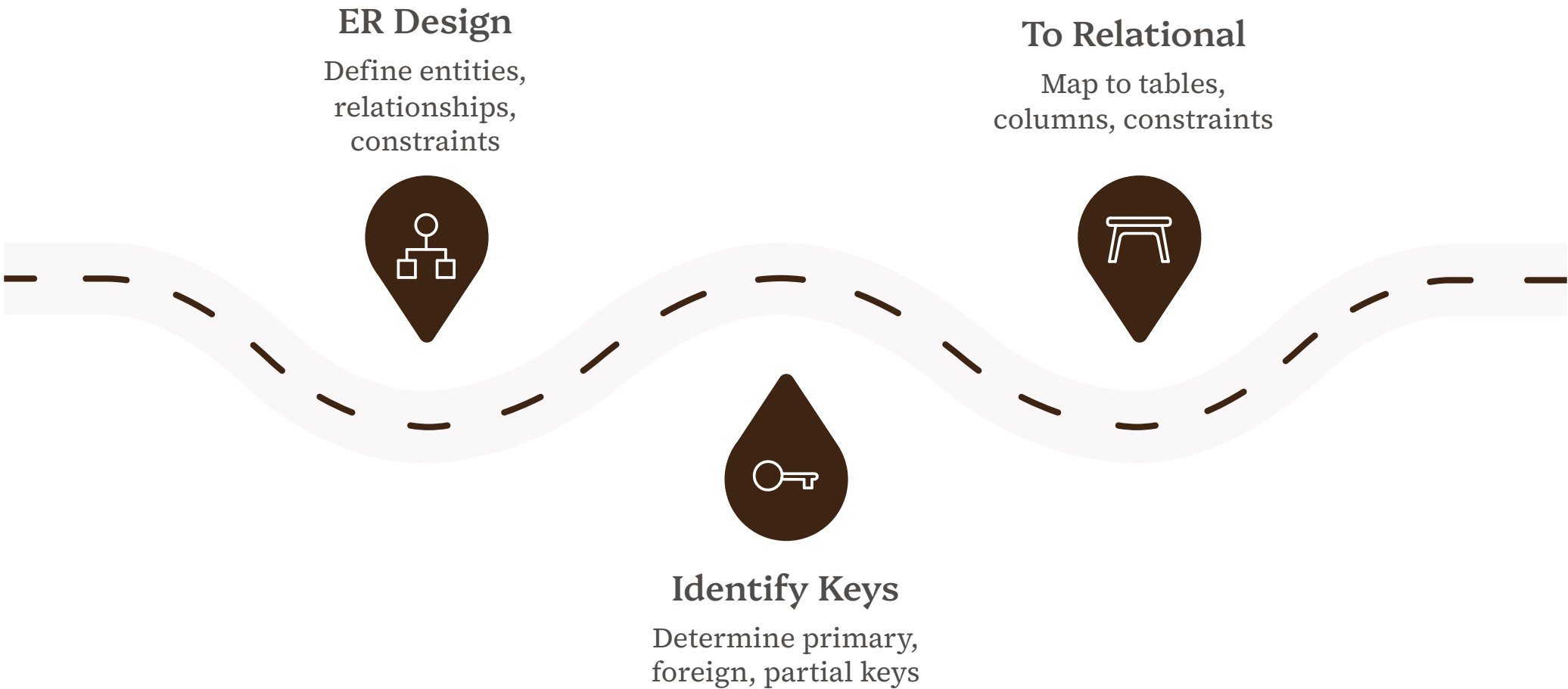
Naming Conventions

Use singular nouns for entities, descriptive verbs for relationships, and avoid redundancy in attribute names. Consistency across the entire schema is non-negotiable.



Keys

Superkey → Candidate Key → Primary Key → Alternate Key. Choose keys that are unique, stable, minimal, and non-null. Prefer surrogate keys for stability; enforce natural uniqueness with UNIQUE constraints.



In the next lecture, we will translate our ER diagrams into relational schemas — mapping entities to tables, relationships to foreign keys, and constraints to SQL integrity rules.